Name: Nthabiseng Sithole

Brightlearn

Assignment Overview

In this assignment, I worked with a realistic streaming-service dataset. I first created five tables namely users, plans, subscriptions, shows, and viewing_sessions exactly as described in the brief, including all the intentional missing references (like a subscription for a user who doesn't exist). Using Databricks SQL, I then wrote queries to practise the three most important join types: INNER JOIN (Q1–Q5), LEFT JOIN (Q6–Q10), and FULL OUTER JOIN (Q11–Q15). For each question, I followed the required join, selected the exact columns requested, and compared my output with the expected results. Below you'll find my SQL code, screenshots of the results, and my answers to the bonus reasoning questions.

Table 1 users

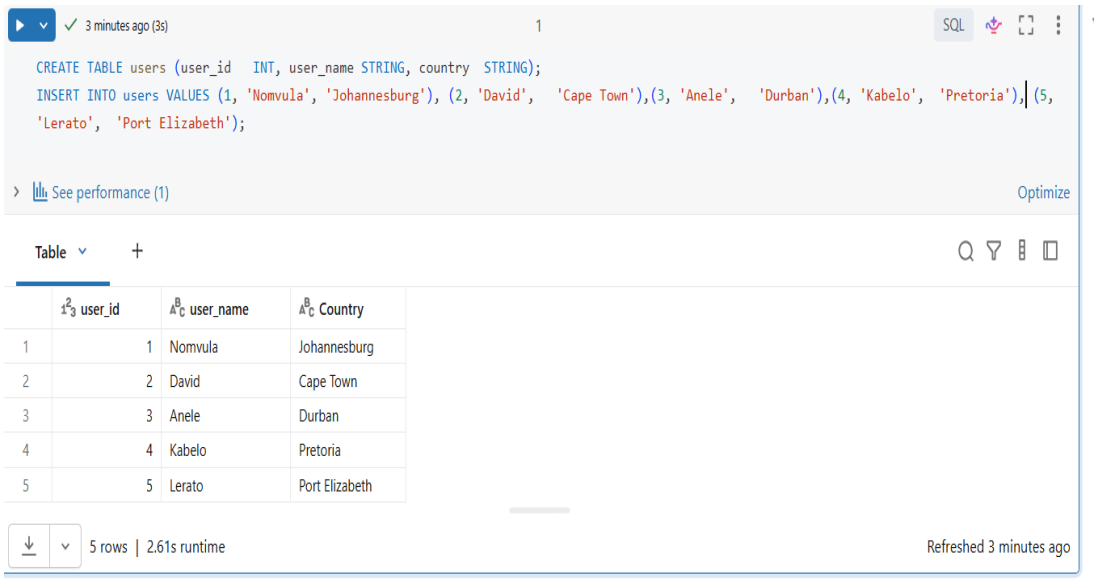


Table 2 plans

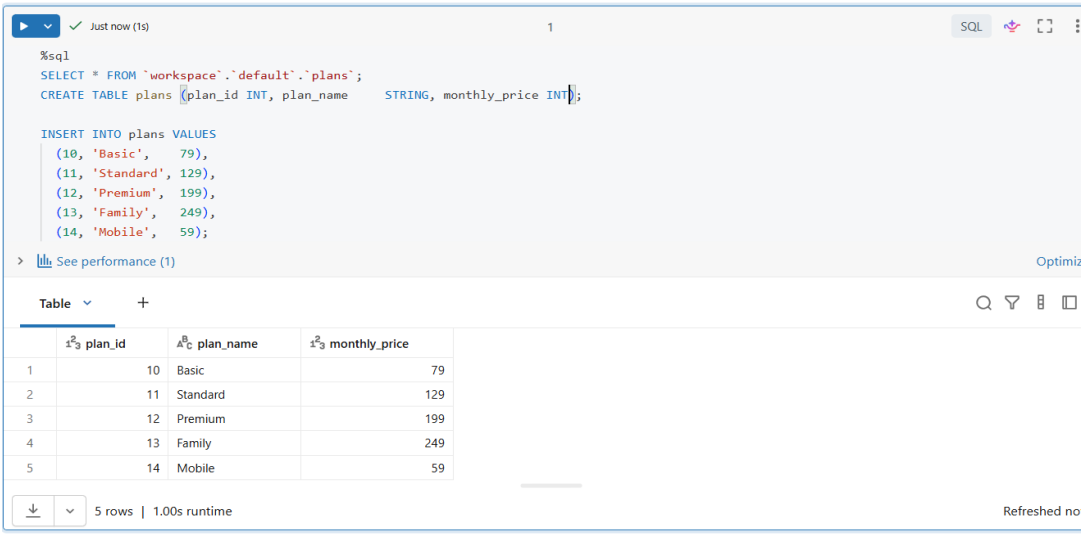


Table 3 Subscriptions

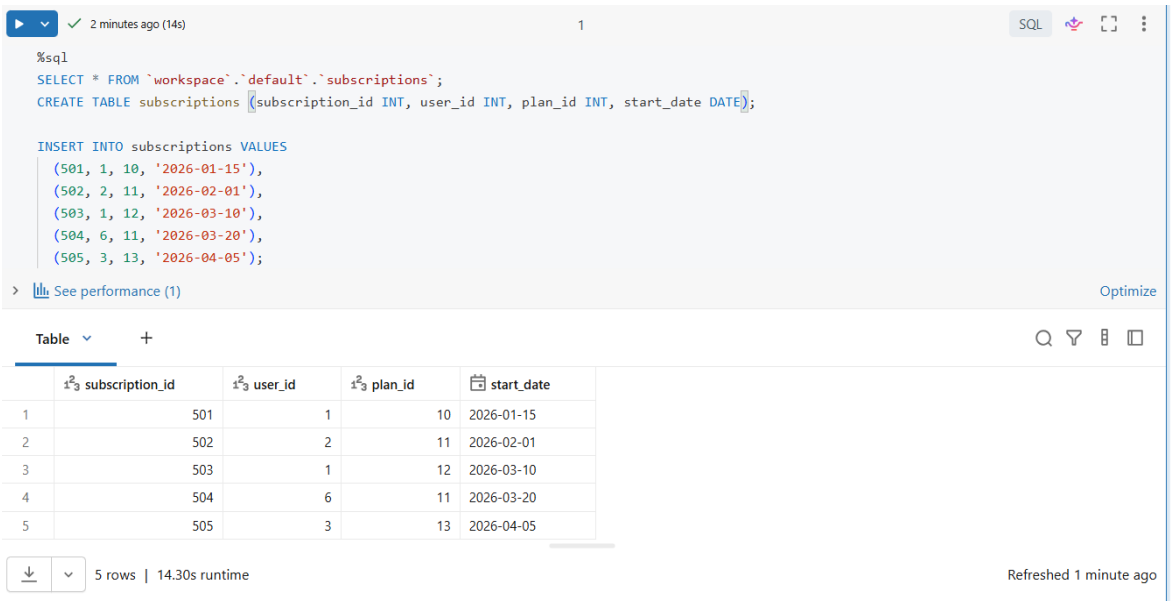


Table 4

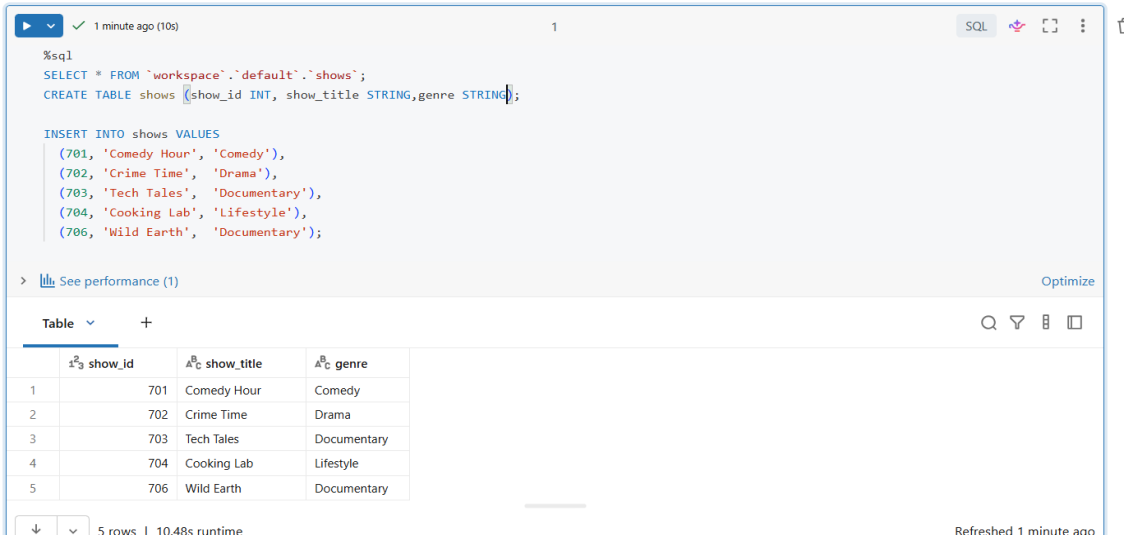
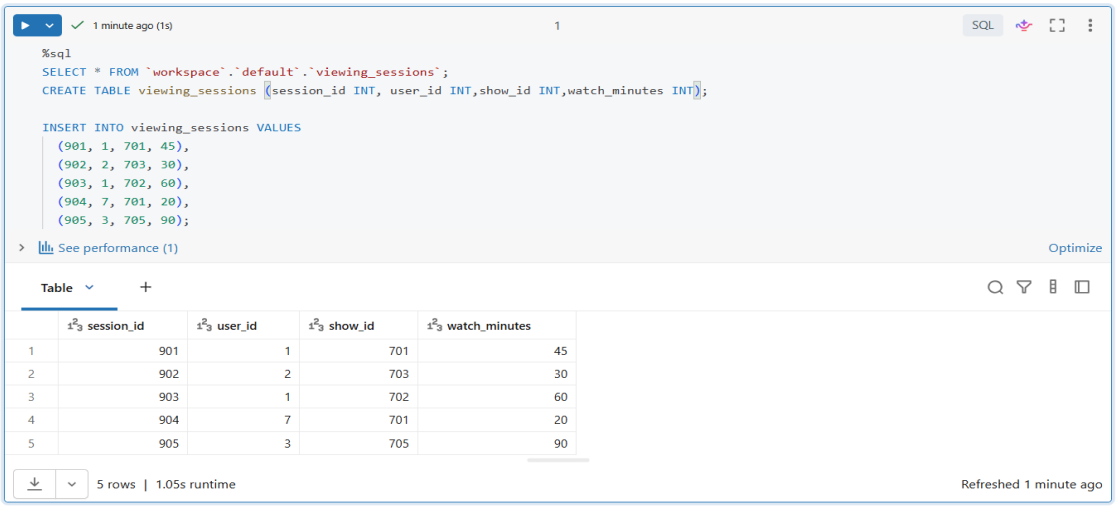
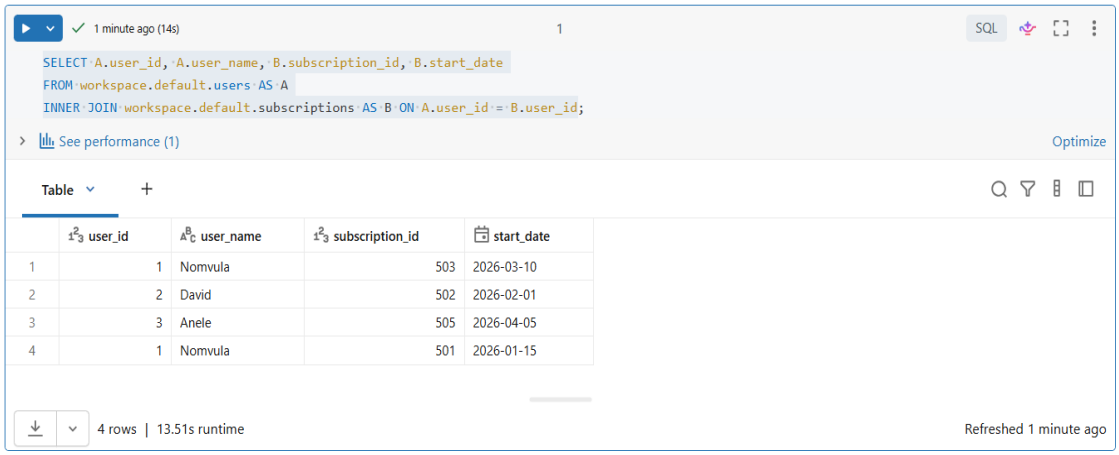


Table 5

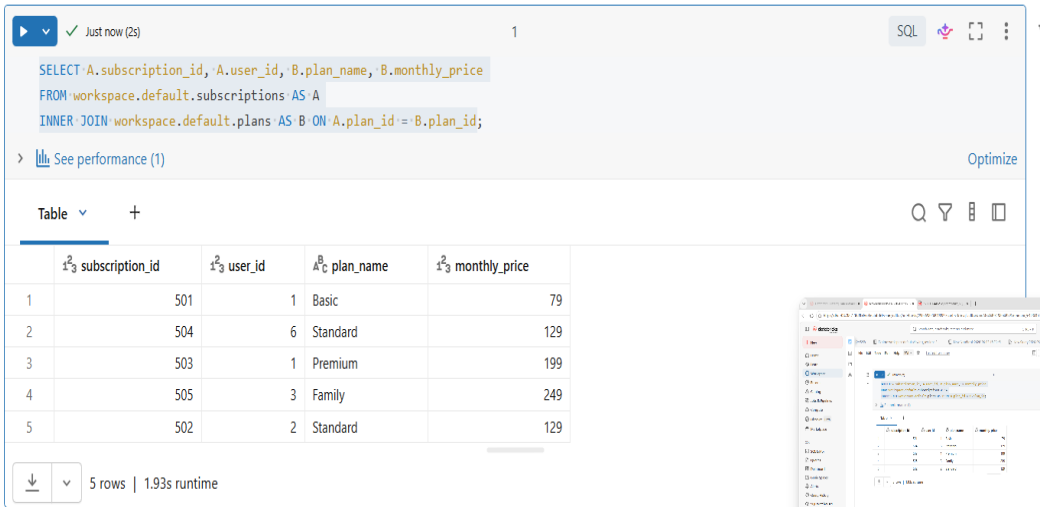


PART A INNER JOIN - For each question: I joined the required tables on the matching key column(s) and returned only rows where both sides have a match

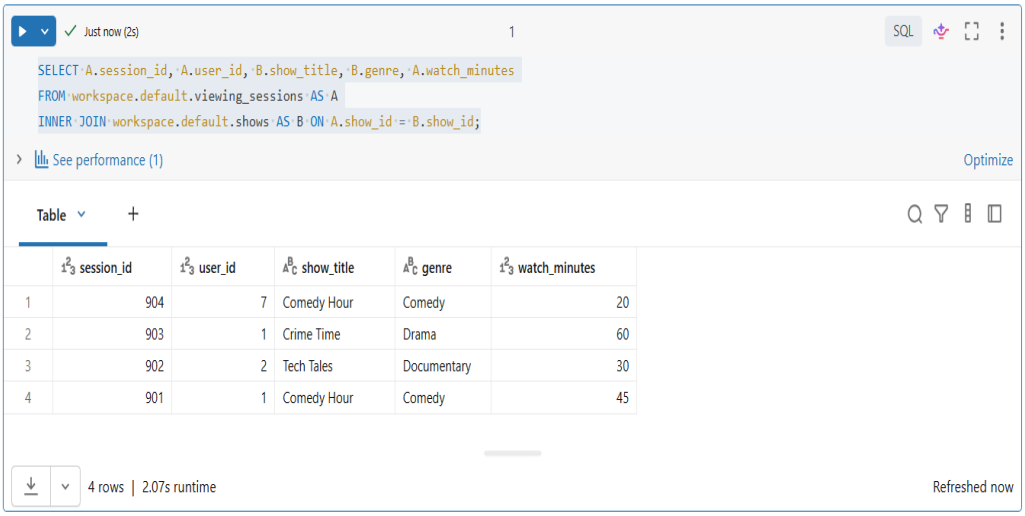
QUESTION 1- Required to show every user who has a subscription. I joined users and subscriptions on user_id



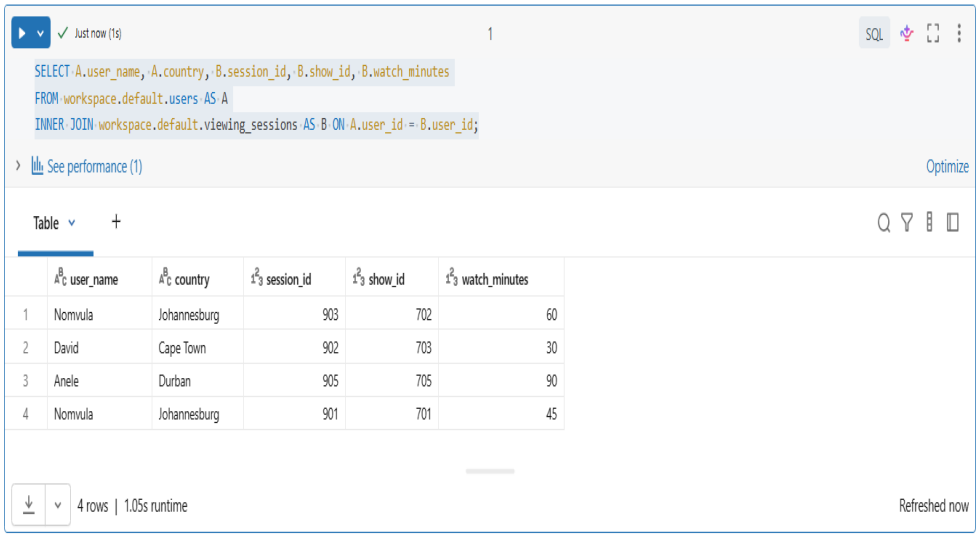
QUESTION 2 - Required to show every subscription with its plan name and monthly price. I joined subscriptions and plans on plan_id.



Question 3 - Required to show every viewing session that has a matching show, including title and genre. I joined viewing_sessions and shows on show_id



Question 4- Required to show every viewing session with the user who watched it, but only sessions with a valid user. I joined users and viewing_sessions on user_id



```
SELECT A.user_name, A.country, B.session_id, B.show_id, B.watch_minutes
FROM workspace.default.users AS A
INNER JOIN workspace.default.viewing_sessions AS B ON A.user_id = B.user_id;
```

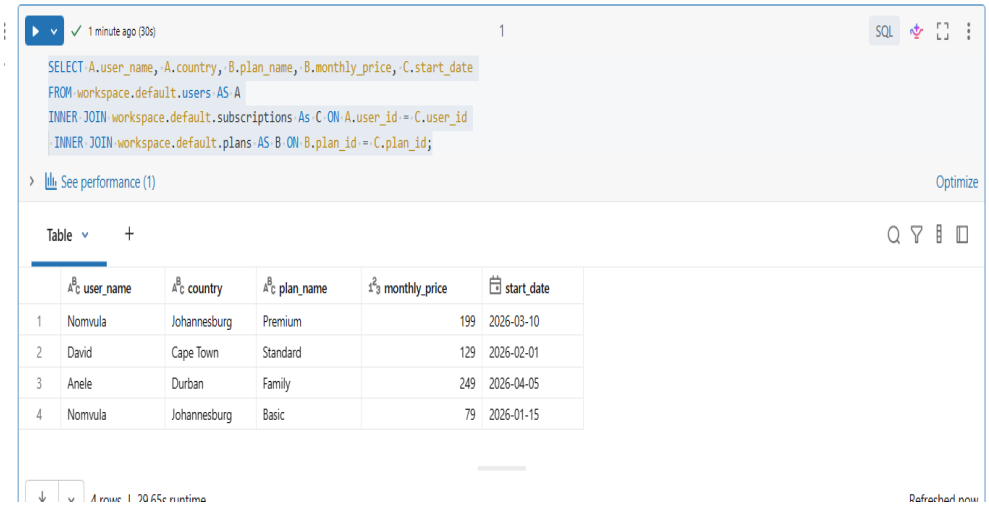
See performance (1) Optimize

Table +

	user_name	country	session_id	show_id	watch_minutes
1	Nomvula	Johannesburg	903	702	60
2	David	Cape Town	902	703	30
3	Anele	Durban	905	705	90
4	Nomvula	Johannesburg	901	701	45

4 rows | 1.05s runtime Refreshed now

Question 5- Required to show users along with their subscription, plan name, and price, but only users who have both a subscription and a valid plan I joined users → subscriptions on user_id, then subscriptions → plans on plan_id (both INNER JOINS).



```
SELECT A.user_name, A.country, B.plan_name, B.monthly_price, C.start_date
FROM workspace.default.users AS A
INNER JOIN workspace.default.subscriptions As C ON A.user_id = C.user_id
INNER JOIN workspace.default.plans AS B ON B.plan_id = C.plan_id;
```

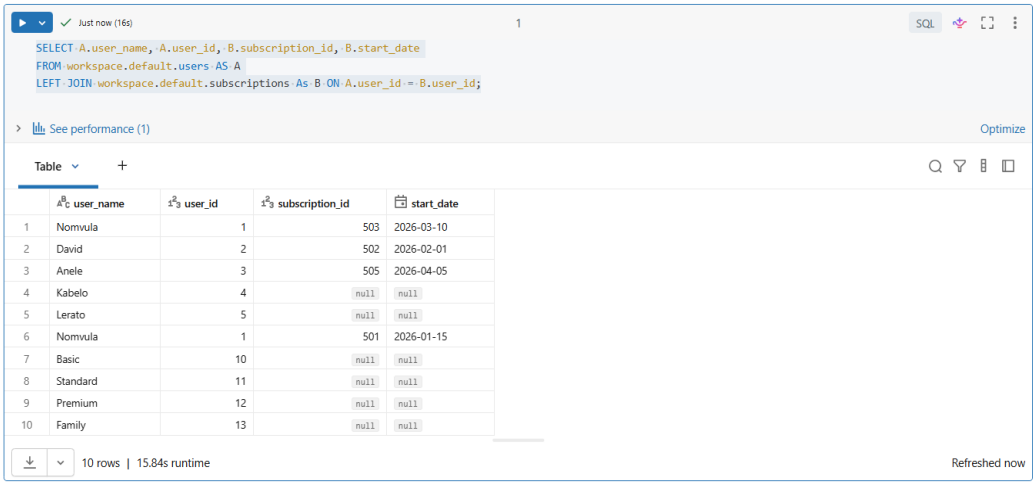
See performance (1) Optimize

Table +

	user_name	country	plan_name	monthly_price	start_date
1	Nomvula	Johannesburg	Premium	199	2026-03-10
2	David	Cape Town	Standard	129	2026-02-01
3	Anele	Durban	Family	249	2026-04-05
4	Nomvula	Johannesburg	Basic	79	2026-01-15

PART B LEFT JOIN- For each question: I kept every row from the left table (the first one mentioned in the requirement), filling unmatched right-side columns with NULL.

QUESTION 6- Required to show every user and any subscriptions they have; users without subscriptions must still appear. I left-joined users to subscriptions on user_id.

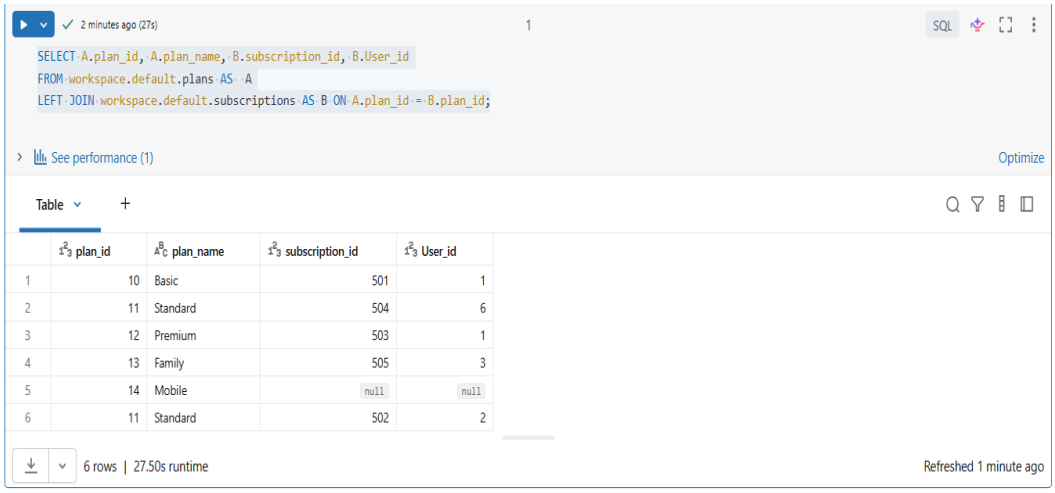


```
SELECT A.user_name, A.user_id, B.subscription_id, B.start_date
FROM workspace.default.users AS A
LEFT JOIN workspace.default.subscriptions As B ON A.user_id = B.user_id;
```

	user_name	user_id	subscription_id	start_date
1	Nomvula	1	503	2026-03-10
2	David	2	502	2026-02-01
3	Anele	3	505	2026-04-05
4	Kabelo	4	null	null
5	Lerato	5	null	null
6	Nomvula	1	501	2026-01-15
7	Basic	10	null	null
8	Standard	11	null	null
9	Premium	12	null	null
10	Family	13	null	null

10 rows | 15.84s runtime — Refreshed now

Question 7- Required to show every plan and the subscriptions on it; plans with no subscribers must still appear. I left-joined plans to subscriptions on plan_id

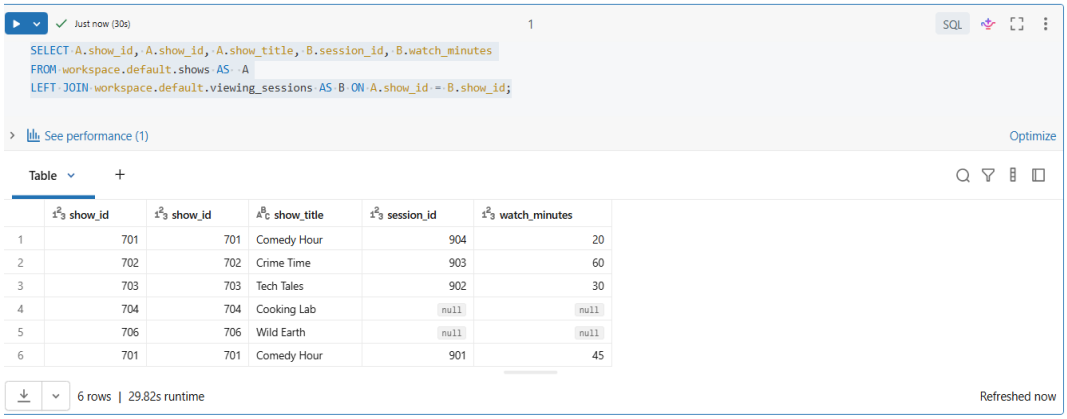


```
SELECT A.plan_id, A.plan_name, B.subscription_id, B.User_id
FROM workspace.default.plans AS A
LEFT JOIN workspace.default.subscriptions AS B ON A.plan_id = B.plan_id;
```

	plan_id	plan_name	subscription_id	User_id
1	10	Basic	501	1
2	11	Standard	504	6
3	12	Premium	503	1
4	13	Family	505	3
5	14	Mobile	null	null
6	11	Standard	502	2

6 rows | 27.50s runtime — Refreshed 1 minute ago

Question 8- Required to show every show and any viewing sessions; shows that were never watched must still appear. I left-joined shows to viewing_sessions on show_id.



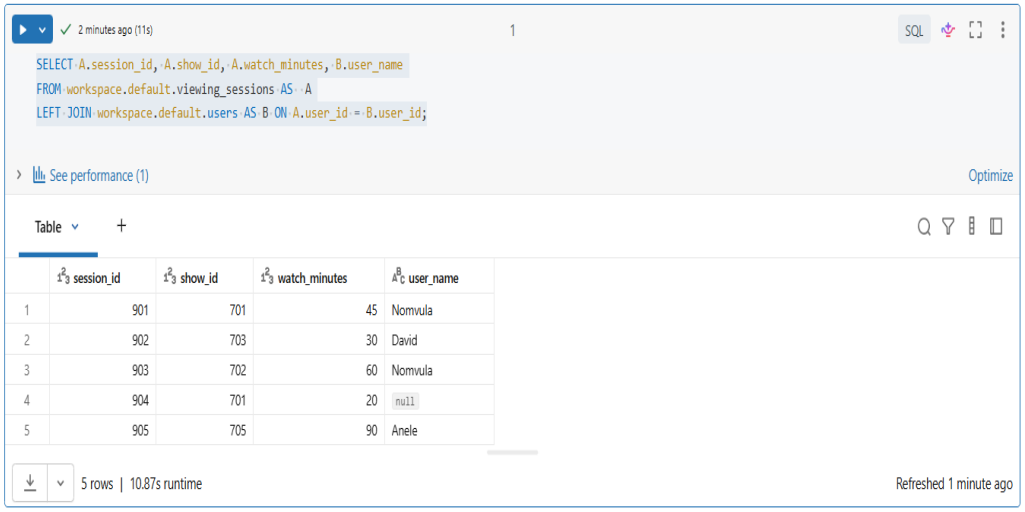
```
SELECT A.show_id, A.show_id, A.show_title, B.session_id, B.watch_minutes
FROM workspace.default.shows AS A
LEFT JOIN workspace.default.viewing_sessions AS B ON A.show_id = B.show_id;
```

	show_id	show_id	show_title	session_id	watch_minutes
1	701	701	Comedy Hour	904	20
2	702	702	Crime Time	903	60
3	703	703	Tech Tales	902	30
4	704	704	Cooking Lab	null	null
5	706	706	Wild Earth	null	null
6	701	701	Comedy Hour	901	45

6 rows | 29.82s runtime — Refreshed now

Question 9- Required to show every viewing session and the user who watched it; sessions referencing non-existent users must still appear (with NULL user details).

I left-joined viewing_sessions to users on user_id.



```
SELECT A.session_id, A.show_id, A.watch_minutes, B.user_name
FROM workspace.default.viewing_sessions AS A
LEFT JOIN workspace.default.users AS B ON A.user_id = B.user_id;
```

2 minutes ago (11s)

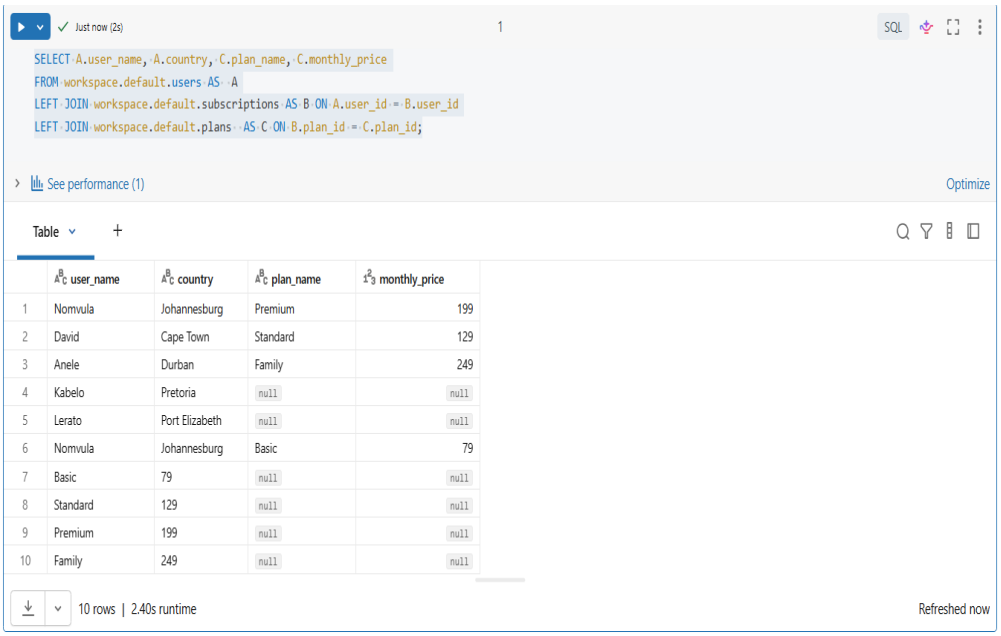
See performance (1) Optimize

Table +

	session_id	show_id	watch_minutes	user_name
1	901	701	45	Nomvula
2	902	703	30	David
3	903	702	60	Nomvula
4	904	701	20	null
5	905	705	90	Anele

5 rows | 10.87s runtime Refreshed 1 minute ago

Question 10- Required to show every user, the plan they are on (if any), and the monthly price; users without a subscription must still appear. I left-joined users to subscriptions on user_id, then left-joined subscriptions to plans on plan_id.



```
SELECT A.user_name, A.country, C.plan_name, C.monthly_price
FROM workspace.default.users AS A
LEFT JOIN workspace.default.subscriptions AS B ON A.user_id = B.user_id
LEFT JOIN workspace.default.plans AS C ON B.plan_id = C.plan_id;
```

Just now (2s)

See performance (1) Optimize

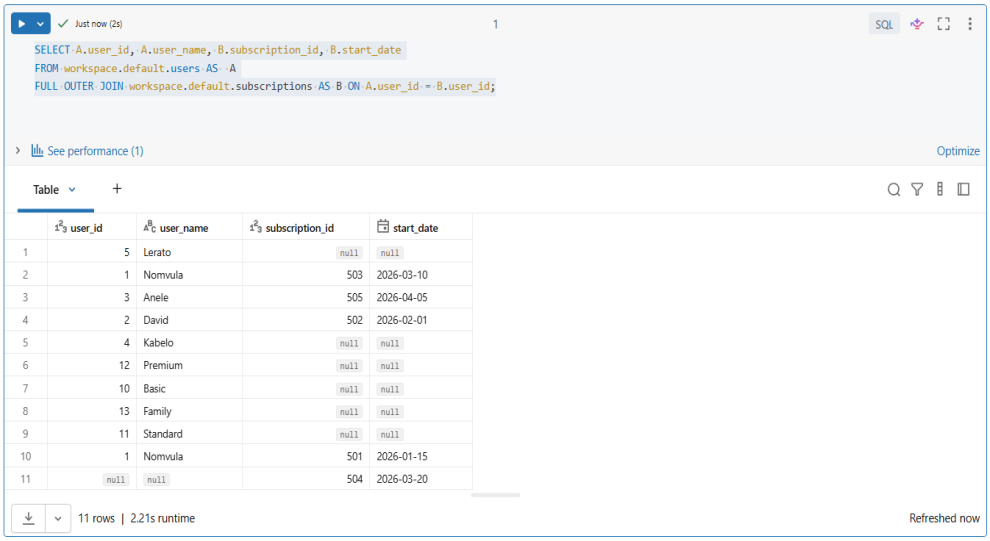
Table +

	user_name	country	plan_name	monthly_price
1	Nomvula	Johannesburg	Premium	199
2	David	Cape Town	Standard	129
3	Anele	Durban	Family	249
4	Kabelo	Pretoria	null	null
5	Lerato	Port Elizabeth	null	null
6	Nomvula	Johannesburg	Basic	79
7	Basic	79	null	null
8	Standard	129	null	null
9	Premium	199	null	null
10	Family	249	null	null

10 rows | 2.40s runtime Refreshed now

PART C FULL OUTER JOIN -For each question: I kept all rows from both tables, matched where possible, and NULL elsewhere. This reveals missing data on either side

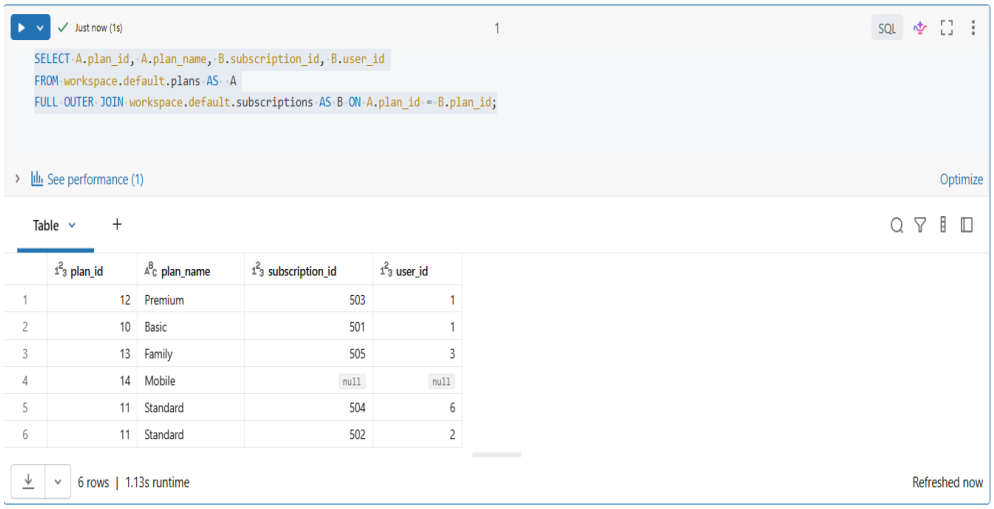
Question 11- Required to show every user and every subscription, including users without subscriptions AND subscriptions referencing users that do not exist. I full outer joined users and subscriptions on user_id.



```
SELECT A.user_id, A.user_name, B.subscription_id, B.start_date
FROM workspace.default.users AS A
FULL OUTER JOIN workspace.default.subscriptions AS B ON A.user_id = B.user_id;
```

	user_id	user_name	subscription_id	start_date
1	5	Lerato	null	null
2	1	Nomvula	503	2026-03-10
3	3	Anele	505	2026-04-05
4	2	David	502	2026-02-01
5	4	Kabelo	null	null
6	12	Premium	null	null
7	10	Basic	null	null
8	13	Family	null	null
9	11	Standard	null	null
10	1	Nomvula	501	2026-01-15
11	null	null	504	2026-03-20

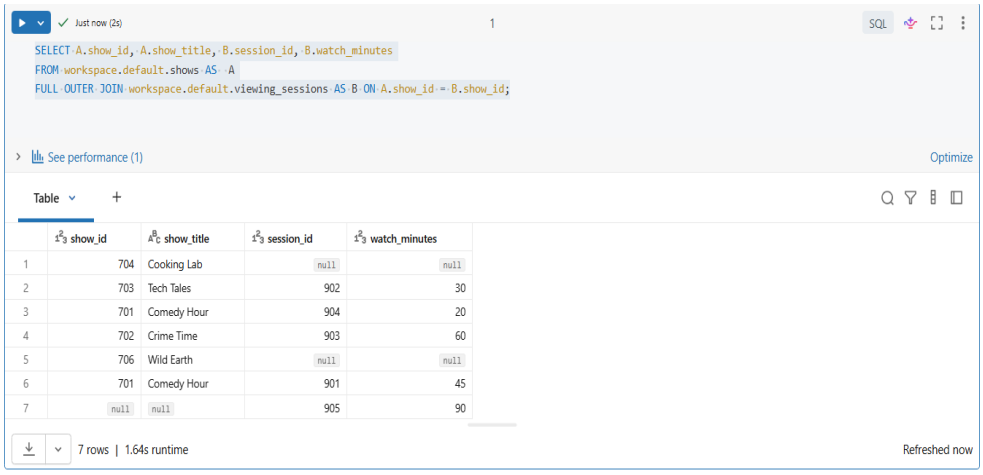
Question 12- Required to show every plan and every subscription, including plans without subscribers AND any subscription referencing a plan that does not exist. I full outer joined plans and subscriptions on plan_id.



```
SELECT A.plan_id, A.plan_name, B.subscription_id, B.user_id
FROM workspace.default.plans AS A
FULL OUTER JOIN workspace.default.subscriptions AS B ON A.plan_id = B.plan_id;
```

	plan_id	plan_name	subscription_id	user_id
1	12	Premium	503	1
2	10	Basic	501	1
3	13	Family	505	3
4	14	Mobile	null	null
5	11	Standard	504	6
6	11	Standard	502	2

Question 13- Required to show every show and every viewing session, including shows never watched AND sessions referencing shows that do not exist. I full outer joined shows and viewing_sessions on show_id.

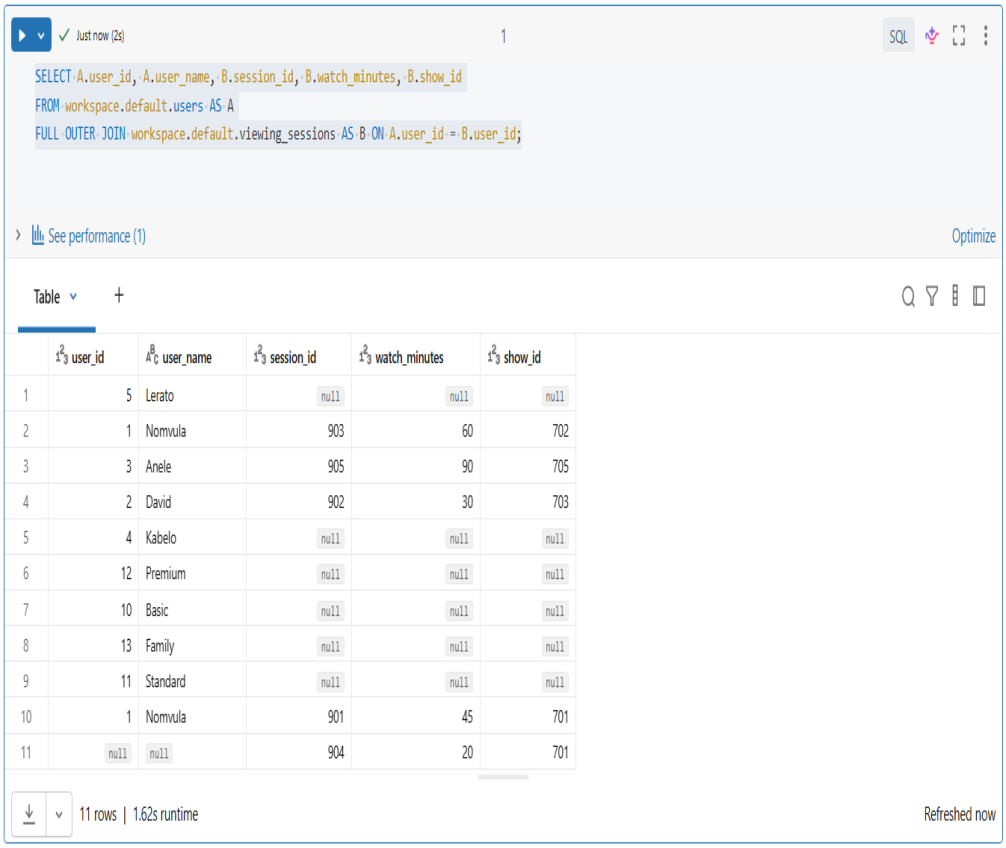


```
SELECT A.show_id, A.show_title, B.session_id, B.watch_minutes
FROM workspace.default.shows AS A
FULL OUTER JOIN workspace.default.viewing_sessions AS B ON A.show_id = B.show_id;
```

	show_id	show_title	session_id	watch_minutes
1	704	Cooking Lab	null	null
2	703	Tech Tales	902	30
3	701	Comedy Hour	904	20
4	702	Crime Time	903	60
5	706	Wild Earth	null	null
6	701	Comedy Hour	901	45
7	null	null	905	90

7 rows | 1.64s runtime

Question 14- Required to show every user and every viewing session, including users with no sessions AND sessions referencing users who do not exist. I full outer joined users and viewing_sessions on user_id.

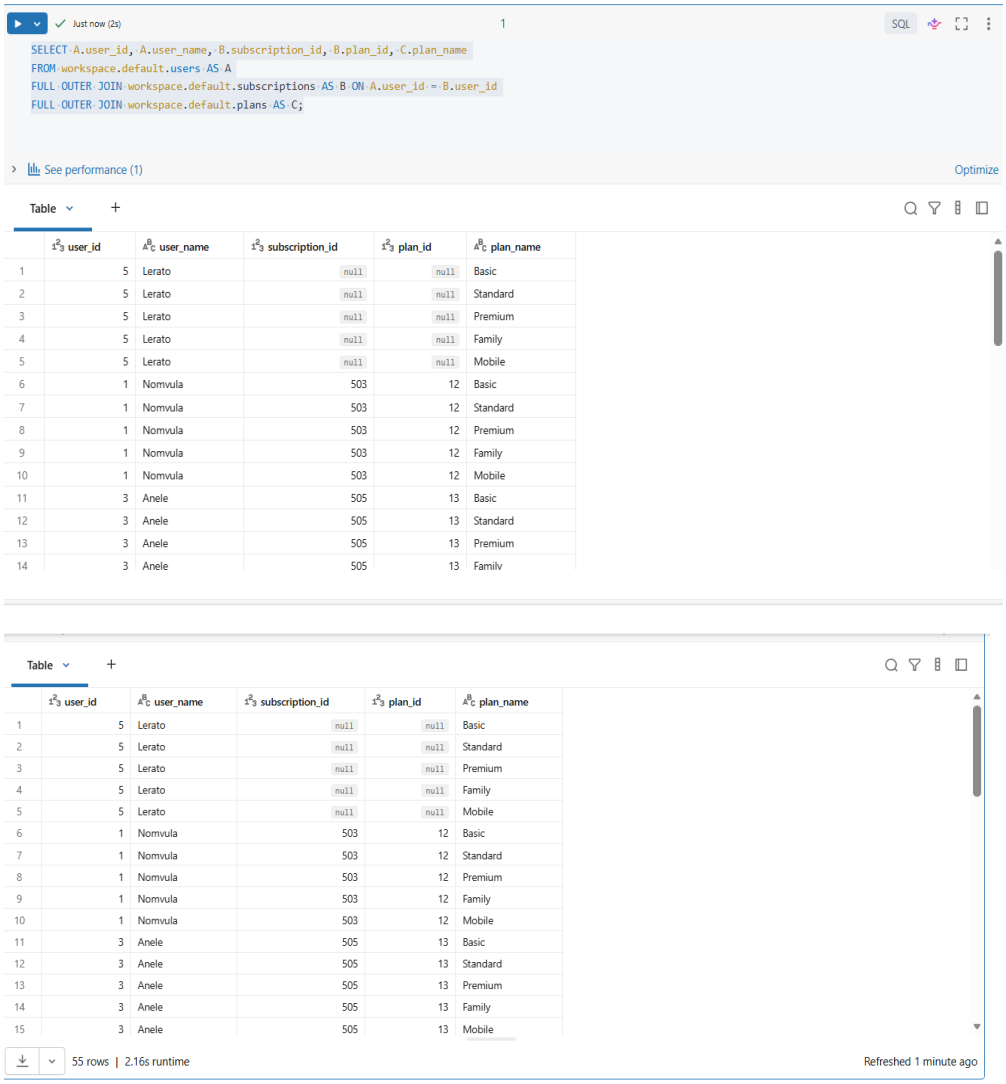


```
SELECT A.user_id, A.user_name, B.session_id, B.watch_minutes, B.show_id
FROM workspace.default.users AS A
FULL OUTER JOIN workspace.default.viewing_sessions AS B ON A.user_id = B.user_id;
```

	user_id	user_name	session_id	watch_minutes	show_id
1	5	Lerato	null	null	null
2	1	Nomvula	903	60	702
3	3	Anele	905	90	705
4	2	David	902	30	703
5	4	Kabelo	null	null	null
6	12	Premium	null	null	null
7	10	Basic	null	null	null
8	13	Family	null	null	null
9	11	Standard	null	null	null
10	1	Nomvula	901	45	701
11	null	null	904	20	701

11 rows | 1.62s runtime

Question 15- Required to show every user, every subscription, and every plan in one query, using FULL OUTER JOIN throughout. This is the hardest – get all gaps visible at once. I full outer joined users and subscriptions on user_id, then full outer joined the result with plans on plan_id.



```
SELECT A.user_id, A.user_name, B.subscription_id, B.plan_id, C.plan_name
FROM workspace.default.users AS A
FULL OUTER JOIN workspace.default.subscriptions AS B ON A.user_id = B.user_id
FULL OUTER JOIN workspace.default.plans AS C;
```

Just now (2s) · SQL · See performance (1) · Optimize

Table

	user_id	user_name	subscription_id	plan_id	plan_name
1	5	Lerato	null	null	Basic
2	5	Lerato	null	null	Standard
3	5	Lerato	null	null	Premium
4	5	Lerato	null	null	Family
5	5	Lerato	null	null	Mobile
6	1	Nomvula	503	12	Basic
7	1	Nomvula	503	12	Standard
8	1	Nomvula	503	12	Premium
9	1	Nomvula	503	12	Family
10	1	Nomvula	503	12	Mobile
11	3	Anele	505	13	Basic
12	3	Anele	505	13	Standard
13	3	Anele	505	13	Premium
14	3	Anele	505	13	Family

Table

	user_id	user_name	subscription_id	plan_id	plan_name
1	5	Lerato	null	null	Basic
2	5	Lerato	null	null	Standard
3	5	Lerato	null	null	Premium
4	5	Lerato	null	null	Family
5	5	Lerato	null	null	Mobile
6	1	Nomvula	503	12	Basic
7	1	Nomvula	503	12	Standard
8	1	Nomvula	503	12	Premium
9	1	Nomvula	503	12	Family
10	1	Nomvula	503	12	Mobile
11	3	Anele	505	13	Basic
12	3	Anele	505	13	Standard
13	3	Anele	505	13	Premium
14	3	Anele	505	13	Family
15	3	Anele	505	13	Mobile

55 rows | 2.16s runtime · Refreshed 1 minute ago

Bonus Questions

1. Which users have not subscribed to any plan?
   User 4, Kabelo, User 5, Lerato
2. Which subscriptions reference users that do not exist in the users table?
   User_id =6, subscription 504
3. Which shows have never been watched?
   Shows 704 (Cooking Lab) and 706 (Wild Earth Documentary)
4. Which viewing sessions reference shows that do not exist?
5. Session 905, show_id 705